# Civic Security

## Informe de avance del MVP — backend, frontend y base de datos

Curso ET201, Formación de Empresas de Base Tecnológica I — Universidad Católica San Pablo
Integrantes: *Anthony Briceño, Maycol Canaveri y Marcelo Torres* | 5 de noviembre de 2026

**Índice**

1 Resumen 1
2 Estado del MVP frente al alcance 1
3 Base de datos 1
4 Backend 1
5 Frontend (aplicación móvil Android) 4
6 Diagramas UML 5
7 Pruebas y seguridad 5
8 Hallazgos de la prueba en tablet 5
9 Plan para la próxima semana 8
A Código representativo 8

## 1. Resumen

Civic Security es una red ciudadana para reportar incidentes de seguridad de forma anónima y validarlos de forma comunitaria en Arequipa. El MVP funciona hoy de punta a punta para **ver reportes en un mapa y enviar un reporte anónimo**; la **validación comunitaria**, diferencial del producto, está diseñada y queda para las siguientes iteraciones. Repositorio público: `https://github.com/aaabriceno/Proyecto-MVP-Civic-Security`.

- **Funciona hoy:** mapa centrado en la ubicación del usuario con los reportes del servidor, filtros por tipo, detalle y envío de un reporte anónimo (tipo, ubicación, descripción).
- **Anonimato por diseño:** el reporte no tiene campo de usuario, no se guarda la IP con él y se rechazan datos personales (DNI, teléfonos, correos, enlaces) en la descripción.
- **Calidad y seguridad:** 61 pruebas del backend y 120 de la app en verde; análisis estático y auditoría de dependencias sin hallazgos; revisión con ataques reales al servidor y 12 hallazgos corregidos.

## 2. Estado del MVP frente al alcance

| Funcionalidad | Estado | Nota |
|---|---|---|
| Base de datos con PostGIS y datos de prueba | Hecho | Tabla `incidentes` con índice espacial |
| API para crear reportes y API del mapa | Hecho | `POST /v1/incidents`, `GET /v1/map/snapshot` |
| Canal en tiempo real en el backend | Hecho | La app todavía no lo escucha |
| Mapa con ubicación del usuario, pines por tipo, filtros y detalle | Hecho | Frame 05 de Figma (parcial) |
| Formulario de reporte anónimo con 6 tipos | Hecho | Frames 06, 06c y 07 de Figma |
| Seguridad básica de la API y de la app | Hecho | Sección 7 |
| Pines nuevos en tiempo real en la app | Pendiente | Backend listo; falta la app |
| Cuentas, inicio de sesión y verificación por DNI | Pendiente | Frames 01–04 y 10 |
| **Validación comunitaria (3 confirmaciones a $<$ 1 km)** | Pendiente | **Prioridad n.º 1 del equipo** |
| Mapa de calor, alertas por zona y foto con rostros difuminados | Pendiente | Frames 05b, 09, 06a y 06b |

## 3. Base de datos

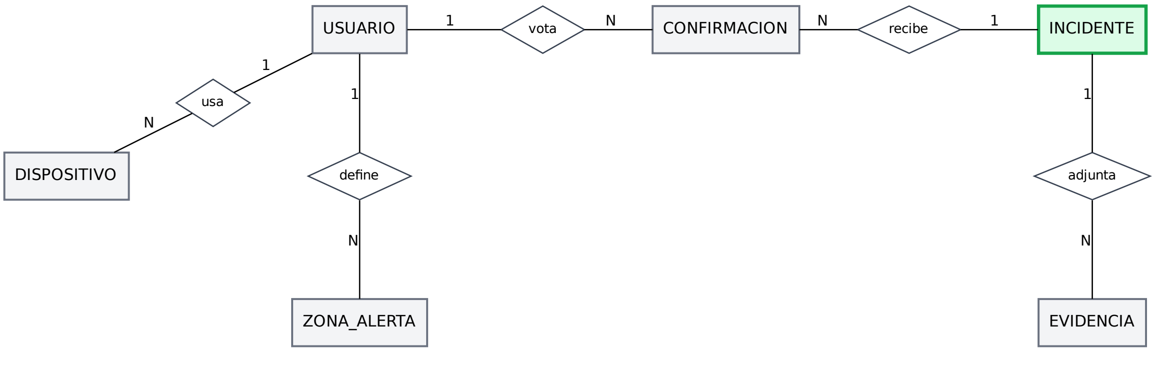
El motor es **PostgreSQL 17 con PostGIS 3.5** en un contenedor Docker (`infra/docker-compose.yml`) que solo acepta conexiones locales. PostGIS resuelve dentro de la base de datos, con índices, las consultas geográficas del producto: reportes del área visible, reportes a menos de 1 km (regla de votación) y el futuro mapa de calor. El diseño se documenta en tres niveles (conceptual, lógico y físico); **verde = implementado, gris = planificado**.

**Decisiones de diseño:** identificadores **UUID** (los reportes no se pueden enumerar, OWASP API1:2023); `geography(Point,4326)` (distancias reales en metros; PostGIS guarda (longitud, latitud) y la API expone `lat` y `lng` por separado); índice **GiST** sobre `ubicacion` (las búsquedas por rectángulo del mapa no recorren todas las filas); `idempotency_key` **único** (un reintento por mala señal no duplica el reporte); **ninguna columna identifica al usuario**; la verificación por DNI guardará solo un HMAC y no se guardan datos biométricos; el detalle se elimina a los 5 años.

Para la demostración hay 80 reportes **ficticios** (`backend/priv/repo/seeds.exs`), que pasan por las mismas reglas que la API, y consultas de ejemplo (`demo_consultas.sql`): reportes por tipo y estado, filtros de tiempo, reportes a menos de 1 km (`ST_DWithin`) y una agrupación en celdas de unos 500 m.

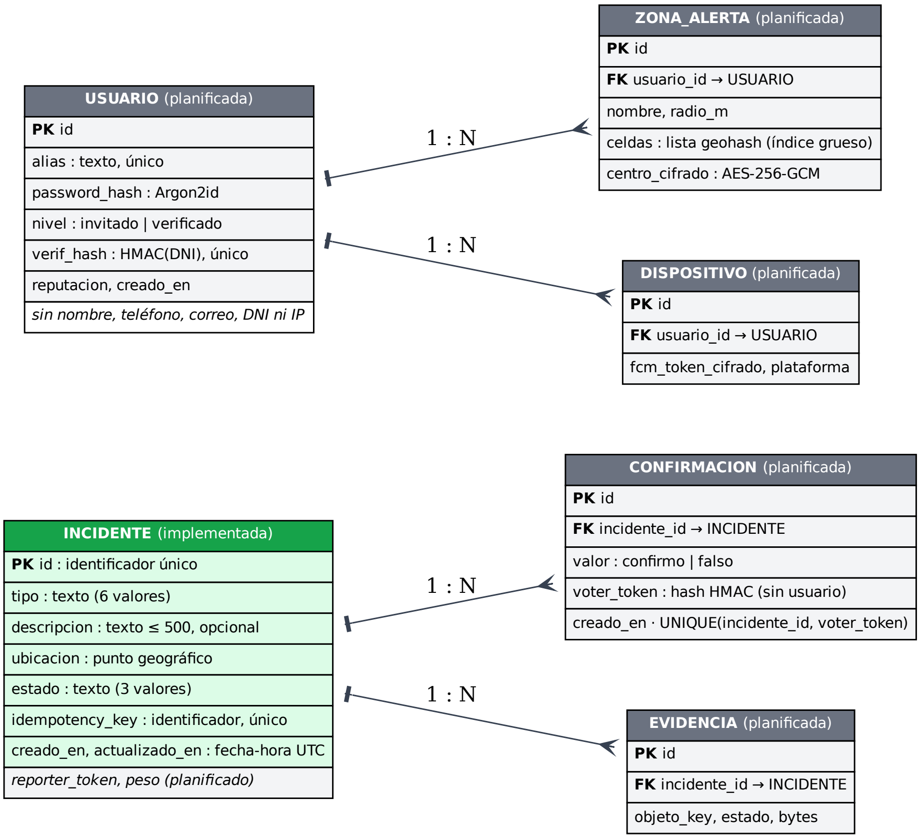
## 4. Backend

Es un **monolito modular** en Elixir con Phoenix: el código se divide en módulos con responsabilidades claras (para mantenerlo) pero se despliega como un solo servicio con una sola base de datos (para operarlo y que rinda).



PRIVACIDAD POR DISEÑO: no existe relación USUARIO–INCIDENTE.
El reporte es anónimo; los votos solo guardan un token HMAC por incidente.

**Figura 1:** Modelo conceptual. Decisión de privacidad: **no existe relación entre USUARIO e INCIDENTE**.



**ZONA_ALERTA** (planificada)
**PK** id
**FK** usuario_id → USUARIO
nombre, radio_m
celdas : lista geohash (índice grueso)
centro_cifrado : AES-256-GCM

**USUARIO** (planificada)
**PK** id
alias : texto, único
password_hash : Argon2id
nivel : invitado | verificado
verif_hash : HMAC(DNI), único
reputacion, creado_en
*sin nombre, teléfono, correo, DNI ni IP*

**DISPOSITIVO** (planificada)
**PK** id
**FK** usuario_id → USUARIO
fcm_token_cifrado, plataforma

**CONFIRMACION** (planificada)
**PK** id
**FK** incidente_id → INCIDENTE
valor : confirmo | falso
voter_token : hash HMAC (sin usuario)
creado_en · UNIQUE(incidente_id, voter_token)

**INCIDENTE** (implementada)
**PK** id : identificador único
tipo : texto (6 valores)
descripcion : texto ≤ 500, opcional
ubicacion : punto geográfico
estado : texto (3 valores)
idempotency_key : identificador, único
creado_en, actualizado_en : fecha-hora UTC
*reporter_token, peso (planificado)*

**EVIDENCIA** (planificada)
**PK** id
**FK** incidente_id → INCIDENTE
objeto_key, estado, bytes

**Figura 2:** Modelo lógico: atributos, claves primarias (PK) y foráneas (FK).

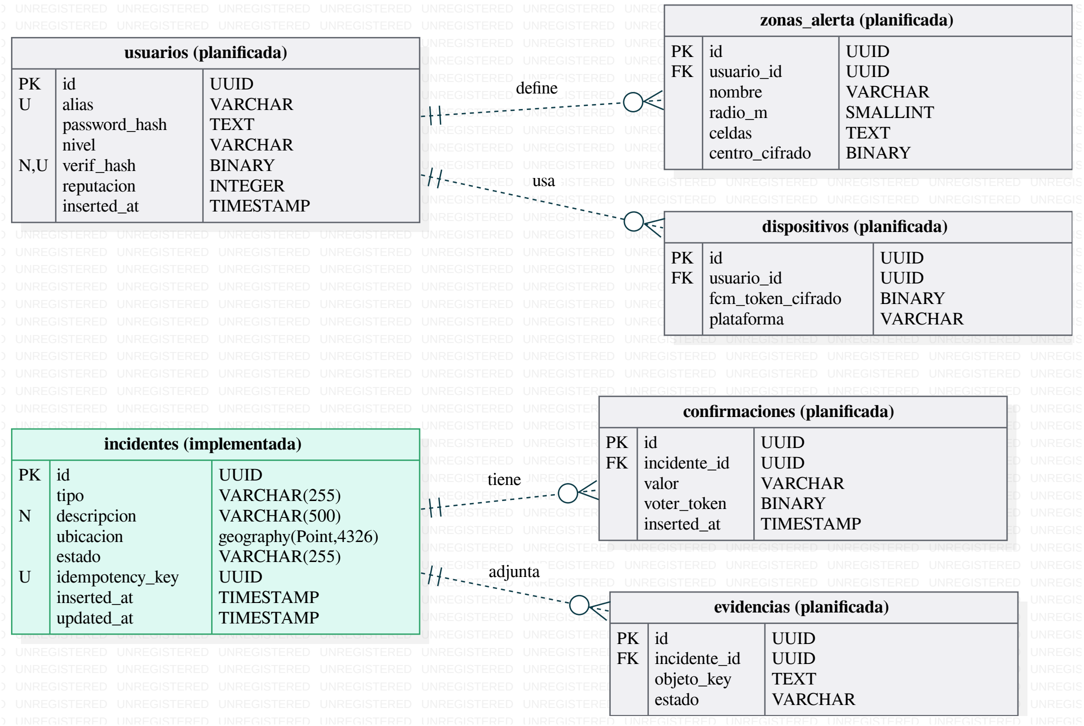

zonas_alerta (planificada)
PK id UUID
FK usuario_id UUID
nombre VARCHAR
radio_m SMALLINT
celdas TEXT
centro_cifrado BINARY
usuarios (planificada)
PK id UUID
U alias VARCHAR
password_hash TEXT
nivel VARCHAR
N,U verif_hash BINARY
reputacion INTEGER
inserted_at TIMESTAMP
define
usa
dispositivos (planificada)
PK id UUID
FK usuario_id UUID
fcm_token_cifrado BINARY
plataforma VARCHAR
confirmaciones (planificada)
PK id UUID
FK incidente_id UUID
valor VARCHAR
voter_token BINARY
inserted_at TIMESTAMP
incidentes (implementada)
PK id UUID
tipo VARCHAR(255)
N descripcion VARCHAR(500)
ubicacion geography(Point,4326)
estado VARCHAR(255)
U idempotency_key UUID
inserted_at TIMESTAMP
updated_at TIMESTAMP
tiene
adjunta
evidencias (planificada)
PK id UUID
FK incidente_id UUID
objeto_key TEXT
estado VARCHAR

**Figura 3:** Modelo físico (StarUML): tipos de PostgreSQL. U = único, N = admite nulos. La tabla `incidentes` es la real (migración `20261027172454_create_incidentes.exs`).

La capa web (`lib/civic_web`) solo recibe peticiones; las reglas viven en el dominio (`lib/civic`). Así, las reglas que distinguen al producto — anonimato, área de Arequipa, no duplicados — se aplican en el servidor y no en la app, que cualquiera puede modificar.

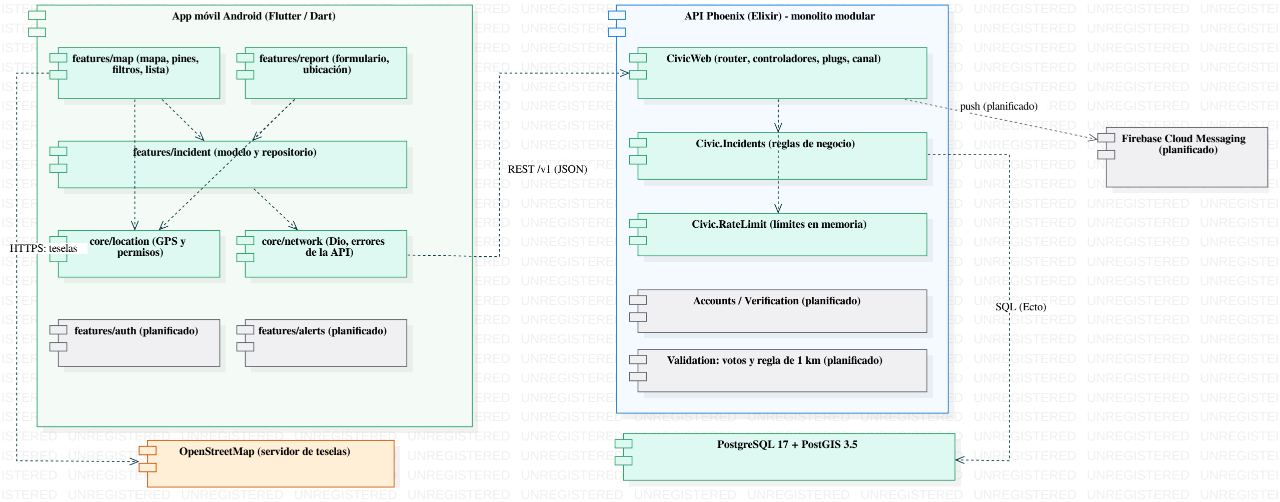

App móvil Android (Flutter / Dart)
features/map (mapa, pines, filtros, lista)
features/report (formulario, ubicación)
features/incident (modelo y repositorio)
core/location (GPS y permisos)
core/network (Dio, errores de la API)
features/auth (planificado)
features/alerts (planificado)
HTTPS: teselas
OpenStreetMap (servidor de teselas)
REST /v1 (JSON)
API Phoenix (Elixir) - monolito modular
CivicWeb (router, controladores, plugs, canal)
Civic.Incidents (reglas de negocio)
Civic.RateLimit (límites en memoria)
Accounts / Verification (planificado)
Validation: votos y regla de 1 km (planificado)
push (planificado)
Firebase Cloud Messaging (planificado)
SQL (Ecto)
PostgreSQL 17 + PostGIS 3.5

**Figura 4:** Diagrama de componentes (StarUML). Verde: implementado; gris: planificado.

## 4.1 API y reglas de negocio

| Endpoint | Respuestas | Descripción |
|---|---|---|
| `GET /health` | 200, 429 | Comprueba que el servidor y la base de datos responden. |
| `POST /v1/incidents` | 201, 200, 400, 409, 413, 415, 422, 429 | Crea un reporte anónimo con el encabezado `idempotency-key`. 200 si se repite el mismo envío; 409 si la clave llega con otro contenido. |
| `GET /v1/map/snapshot` | 200, 400, 429 | Reportes de las últimas 24 h dentro del rectángulo `bbox=minLng,minLat,maxLng,maxLat` (máx. 500). |
| `WS /socket` | — | Tópico `mapa:arequipa`: emite `incidente_nuevo`; los clientes solo escuchan. |

- Seis tipos de incidente; todo reporte nace **"Sin verificar"** y el estado no se puede enviar desde el cliente (lista blanca de campos: `tipo`, `descripcion`, `lat`, `lng`).
- Ubicación obligatoria y dentro de Arequipa; descripción opcional de hasta 500 caracteres, sin caracteres de control ni datos personales; envío **idempotente**.

**Listing 1:** Cadena de validaciones (`backend/lib/civic/incidents/incident.ex`).

```
def create_changeset(incident, attrs) do
  incident
  |> cast(attrs, [:tipo, :descripcion, :lat, :lng])
  |> validate_required([:tipo, :lat, :lng])
  |> update_change(:descripcion, &trim_or_nil/1)
  |> validate_length(:descripcion, max: 500)
  |> validate_texto_seguro(:descripcion)
  |> validate_sin_datos_personales(:descripcion)
  |> validate_arequipa()
  |> put_ubicacion()
  |> unique_constraint(:idempotency_key)
end
```

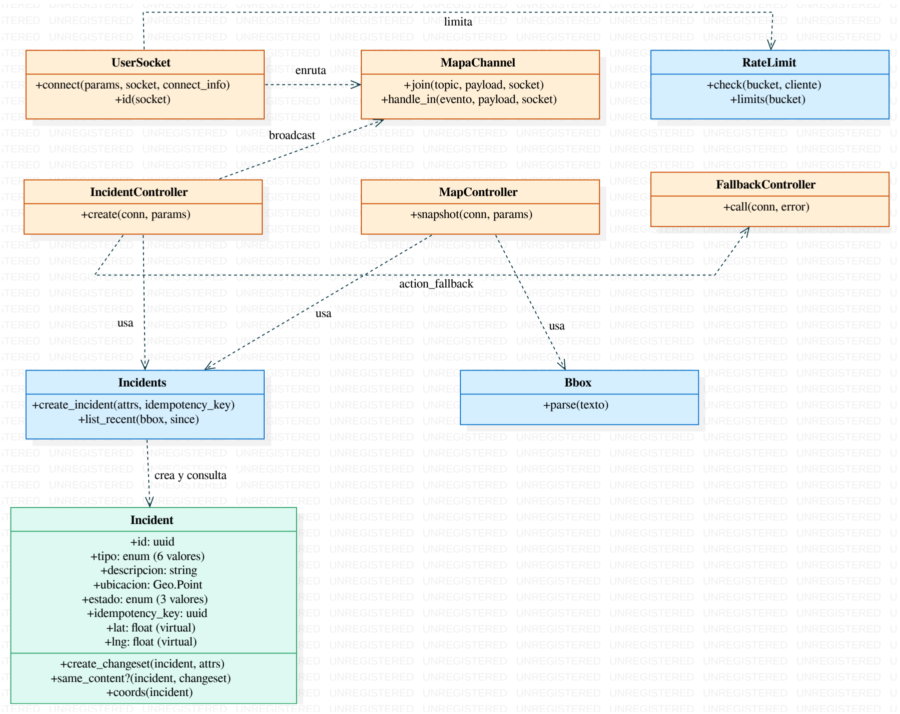

limita
UserSocket
+connect(params, socket, connect_info)
+id(socket)
enruta
MapaChannel
+join(topic, payload, socket)
+handle_in(evento, payload, socket)
RateLimit
+check(bucket, cliente)
+limits(bucket)
broadcast
IncidentController
+create(conn, params)
MapController
+snapshot(conn, params)
FallbackController
+call(conn, error)
action_fallback
usa
usa
usa
Incidents
+create_incident(attrs, idempotency_key)
+list_recent(bbox, since)
Bbox
+parse(texto)
crea y consulta
Incident
+id: uuid
+tipo: enum (6 valores)
+descripcion: string
+ubicacion: Geo.Point
+estado: enum (3 valores)
+idempotency_key: uuid
+lat: float (virtual)
+lng: float (virtual)
+create_changeset(incident, attrs)
+same_content?(incident, changeset)
+coords(incident)

**Figura 5:** Diagrama de clases (módulos) del backend, con sus firmas reales (StarUML).

### 4.2 Evidencia: respuestas reales de la API

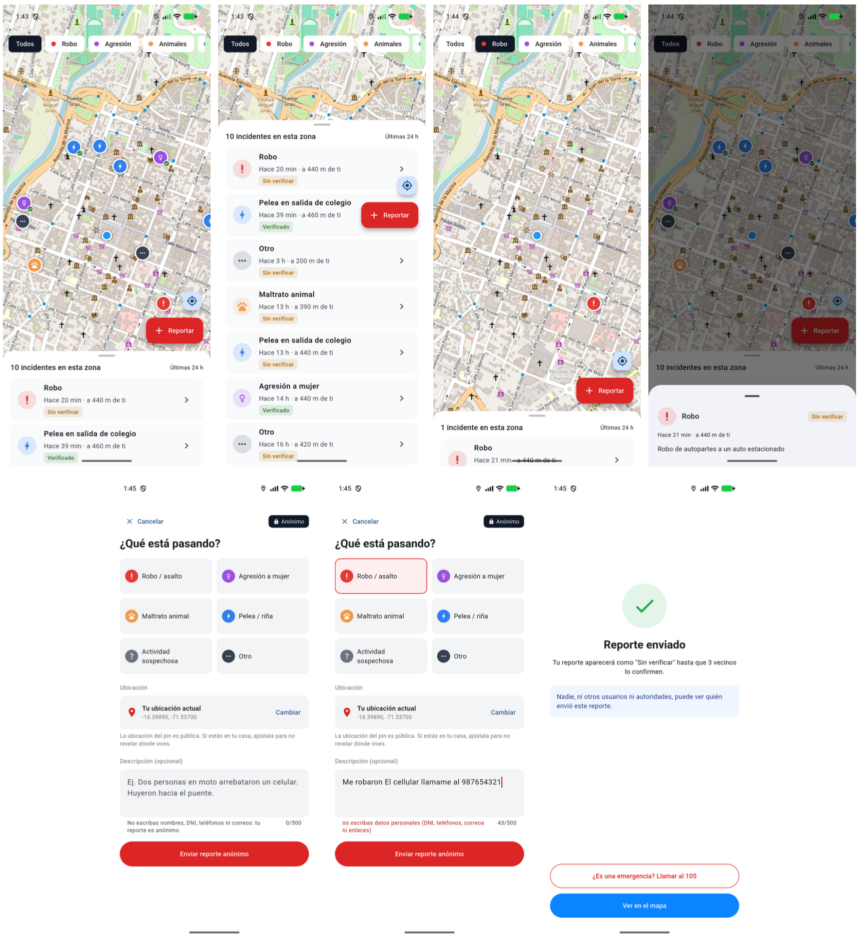
```
$ POST /v1/incidents (robo, lat -16.3963, lng -71.5340; el cliente intenta colar "estado":"verificado")
HTTP/1.1 201 Created {"data":{"id":"a2392f57-...","estado":"sin_verificar","tipo":"robo", ...}} <- se ignora
(mismo envio repetido con la misma clave) ........ HTTP 200
(sin encabezado idempotency-key) ................. HTTP 400 idempotency_key_required
(reporte en Lima) ................................ HTTP 422 "La ubicacion debe estar dentro de Arequipa"
(descripcion con un telefono) .................... HTTP 422 "no escribas datos personales"
(cuerpo mayor a 8 KB) ............................ HTTP 413
```

## 5. Frontend (aplicación móvil Android)

App en **Flutter (Dart)**, probada en Android. Usa **Riverpod** (estado), **flutter_map** con OpenStreetMap (mapa), **geolocator** (ubicación) y **Dio** (API), organizada por funcionalidades (*feature-first*): `core/` (límites de Arequipa, ubicación, red, marcador del 105) y `features/` (`incident`: modelo y repositorio; `map`: mapa, pines, filtros, lista, detalle; `report`: formulario, ajuste de ubicación y confirmación). Se siguen los mockups de Figma (frames 05, 06, 06c y 07). Las capturas se tomaron en un emulador Android contra el backend real, con datos ficticios.

**Comportamientos relevantes:**

- **Ubicación:** permiso en primer plano; el mapa se centra en el usuario. Sin permiso, con el GPS apagado o fuera de Arequipa, muestra Arequipa completa con un aviso; no inventa una ubicación.
- **Carga de datos:** pide solo los reportes del *área visible*, tras 500 ms sin mover el mapa; una respuesta lenta no reemplaza a una más nueva y, ante un error de red, conserva los pines y ofrece “Reintentar”.
- **Envío seguro ante mala señal:** cada formulario genera una clave de idempotencia; reintentar lo mismo no duplica. El botón se deshabilita mientras envía.
- **Privacidad y transporte:** la ubicación del usuario no sale del teléfono; en producción la app exige `https`.



10 incidentes en esta zona — Últimas 24 h

- Robo — Hace 20 min · a 440 m de ti — Sin verificar
- Pelea en salida de colegio — Hace 39 min · a 460 m de ti — Verificado
- Otro — Hace 3 h · a 200 m de ti — Sin verificar
- Maltrato animal — Hace 13 h · a 390 m de ti — Sin verificar
- Pelea en salida de colegio — Hace 13 h · a 440 m de ti — Sin verificar
- Agresión a mujer — Hace 14 h · a 440 m de ti — Verificado
- Otro — Hace 16 h · a 420 m de ti — Sin verificar

1 incidente en esta zona — Últimas 24 h

- Robo — Hace 21 min ~~· a 440 m de ti~~

Robo — Sin verificar
Hace 21 min · a 440 m de ti
Robo de autopartes a un auto estacionado

¿Qué está pasando?

Tu ubicación actual
-16.39890, -71.53700
La ubicación del pin es pública. Si estás en tu casa, ajústala para no revelar dónde vives.

Ej. Dos personas en moto arrebataron un celular. Huyeron hacia el puente.
No escribas nombres, DNI, teléfonos ni correos: tu reporte es anónimo. 0/500

Me robaron El cellular llamame al 987654321
no escribas datos personales (DNI, teléfonos, correos ni enlaces) 43/500

Reporte enviado
Tu reporte aparecerá como "Sin verificar" hasta que 3 vecinos lo confirmen.
Nadie, ni otros usuarios ni autoridades, puede ver quién envió este reporte.

**Figura 6:** Arriba: mapa con pines por tipo, lista “N incidentes en esta zona” con hora y distancia, filtro por tipo (Robo) y detalle. Abajo: formulario con los seis tipos; el servidor bloquea un teléfono escrito en la descripción y la app lo explica; confirmación con “Llamar al 105”.

## 6. Diagramas UML

Elaborados con StarUML. **Verde = implementado; gris punteado = planificado.**

## 7. Pruebas y seguridad

```
$ cd backend && mix test ........ 61 tests, 0 failures $ mix sobelow ........ sin hallazgos
$ mix deps.audit ................ No vulnerabilities found. $ cd app && flutter analyze ... No issues found!
$ flutter test .................. +120: All tests passed!
```

Se trabajó con *pruebas primero*: cada funcionalidad se describe con una prueba que falla y luego se implementa. Además se hizo una revisión actuando como atacante contra el servidor real, antes y después de cada corrección (informe completo en `docs/seguridad-revision-1.md`).

| **Pilar** | **Hallazgo** | **Corrección** |
|---|---|---|
| Disponibilidad | Sin límite de solicitudes (150 reportes seguidos aceptados); cuerpos de 3 MB y formularios aceptados; marcos y conexiones WebSocket sin límite | Límites por cliente (429); solo JSON de hasta 8 KB (413, 415); marco máx. 4 KB y límite de conexiones |
| Integridad | La misma clave con otro contenido perdía el reporte en silencio; APK firmado con la clave de depuración | Responde 409; el release queda sin firmar si falta la clave propia |
| Confidencialidad | Servidor de desarrollo abierto a toda la red y a cualquier origen; sin cabeceras de seguridad | Escucha solo en localhost y verifica el origen; `nosniff`, CSP, `no-store` |

**Riesgos pendientes:** la app usa el superusuario de la base de datos (solo desarrollo; antes del piloto se creará un rol limitado); no hay cuentas, así que cualquiera puede reportar (límites generosos por IP porque muchos alumnos comparten el wifi de la universidad); los reportes con ubicación simulada se tratarán con la validación comunitaria.

## 8. Hallazgos de la prueba en tablet

El 5 de noviembre se instaló la app en una tablet Android por USB contra el backend de la laptop. Funcionó de punta a punta; se investigaron dos observaciones:

1. **Lentitud al cargar y mover el mapa.** *Causa probable:* la prueba se hizo en modo **depuración** de Flutter, entre 3 y 10 veces más lento que la app real, en un dispositivo de gama media. *Descartado:* la descarga de teselas (`flutter_map` las guarda en disco y cancela las innecesarias). *Acción:* se habilitó el modo **profile** para medir

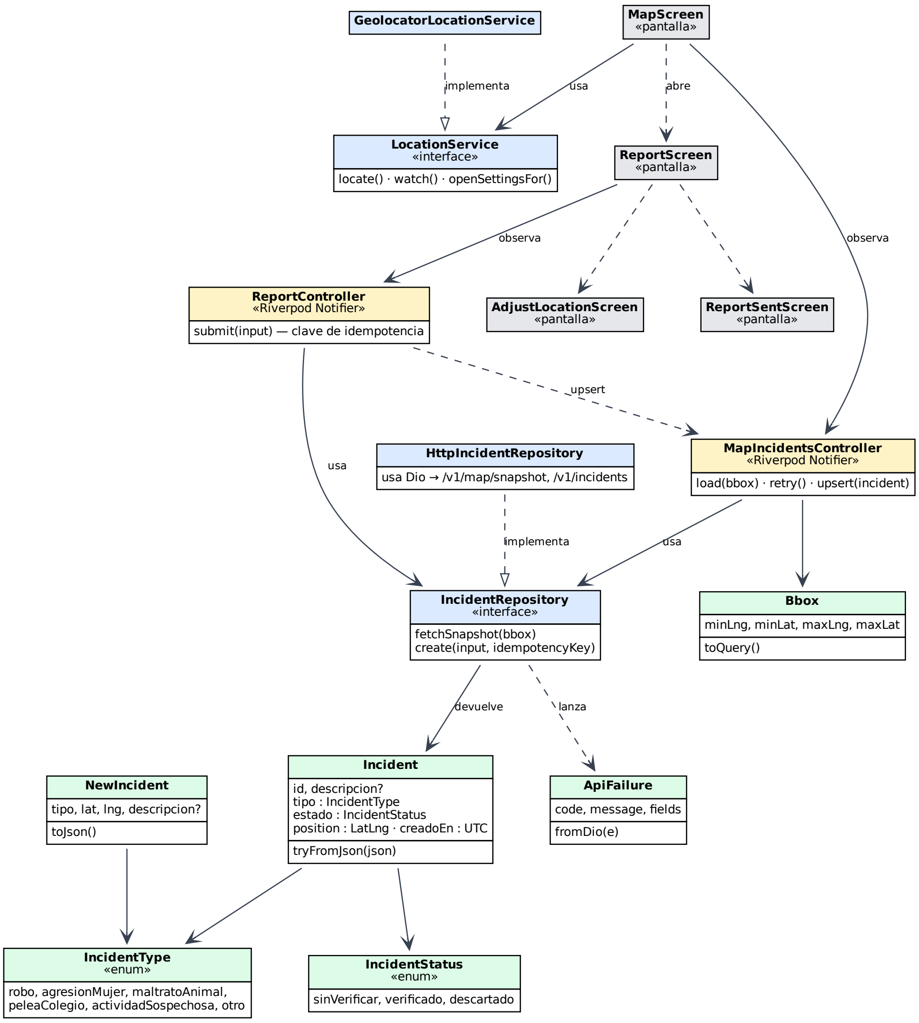

GeolocatorLocationService
MapScreen
«pantalla»
implementa
usa
abre
LocationService
«interface»
locate() · watch() · openSettingsFor()
ReportScreen
«pantalla»
observa
observa
ReportController
«Riverpod Notifier»
submit(input) — clave de idempotencia
AdjustLocationScreen
«pantalla»
ReportSentScreen
«pantalla»
upsert
usa
HttpIncidentRepository
usa Dio → /v1/map/snapshot, /v1/incidents
MapIncidentsController
«Riverpod Notifier»
load(bbox) · retry() · upsert(incident)
implementa
usa
IncidentRepository
«interface»
fetchSnapshot(bbox)
create(input, idempotencyKey)
Bbox
minLng, minLat, maxLng, maxLat
toQuery()
devuelve
lanza
NewIncident
tipo, lat, lng, descripcion?
toJson()
Incident
id, descripcion?
tipo : IncidentType
estado : IncidentStatus
position : LatLng · creadoEn : UTC
tryFromJson(json)
ApiFailure
code, message, fields
fromDio(e)
IncidentType
«enum»
robo, agresionMujer, maltratoAnimal,
peleaColegio, actividadSospechosa, otro
IncidentStatus
«enum»
sinVerificar, verificado, descartado

**Figura 7:** Diagrama de clases de la app: modelo, repositorio, controladores de estado y pantallas.

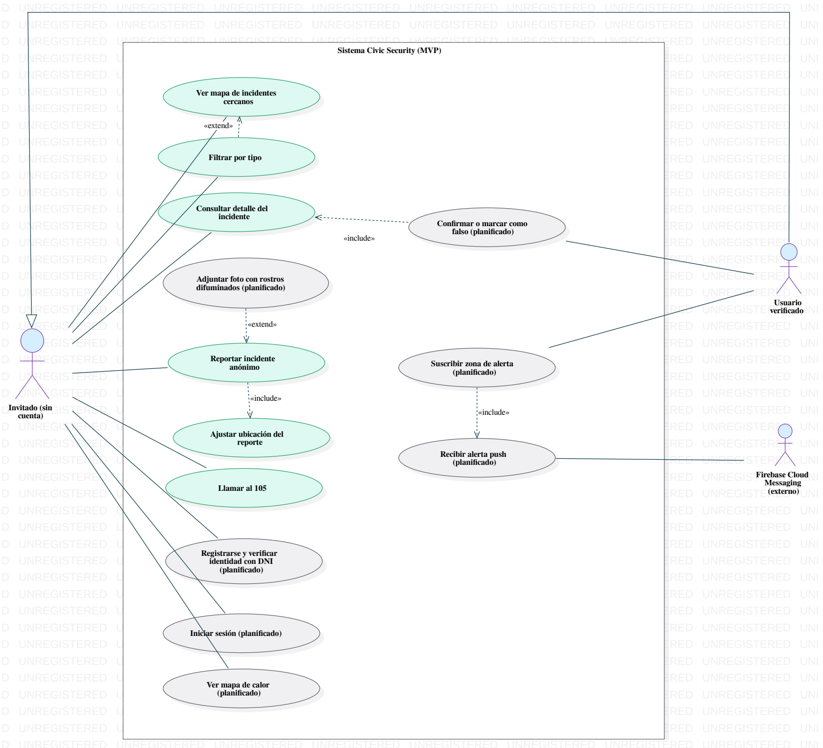

Sistema Civic Security (MVP)
Ver mapa de incidentes cercanos
«extend»
Filtrar por tipo
Consultar detalle del incidente
«include»
Confirmar o marcar como falso (planificado)
Adjuntar foto con rostros difuminados (planificado)
«extend»
Usuario verificado
Reportar incidente anónimo
Suscribir zona de alerta (planificado)
Invitado (sin cuenta)
«include»
«include»
Ajustar ubicación del reporte
Recibir alerta push (planificado)
Firebase Cloud Messaging (externo)
Llamar al 105
Registrarse y verificar identidad con DNI (planificado)
Iniciar sesión (planificado)
Ver mapa de calor (planificado)

**Figura 8:** Casos de uso del MVP. El usuario verificado generaliza al invitado: puede, además, confirmar o desmentir reportes y suscribir zonas de alerta.

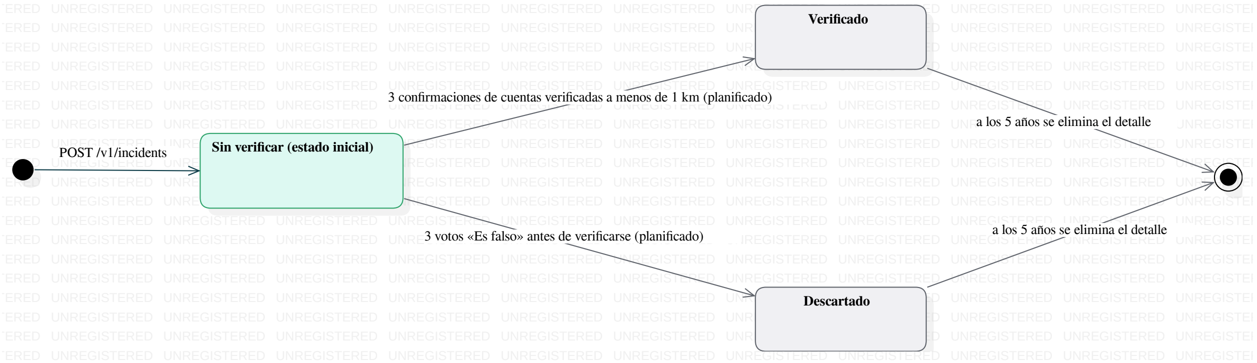

Verificado
3 confirmaciones de cuentas verificadas a menos de 1 km (planificado)
a los 5 años se elimina el detalle
POST /v1/incidents
Sin verificar (estado inicial)
3 votos «Es falso» antes de verificarse (planificado)
a los 5 años se elimina el detalle
Descartado

**Figura 9:** Estados del incidente. La columna `estado` ya admite los tres valores; las transiciones dependen de la validación comunitaria (iteración 2).

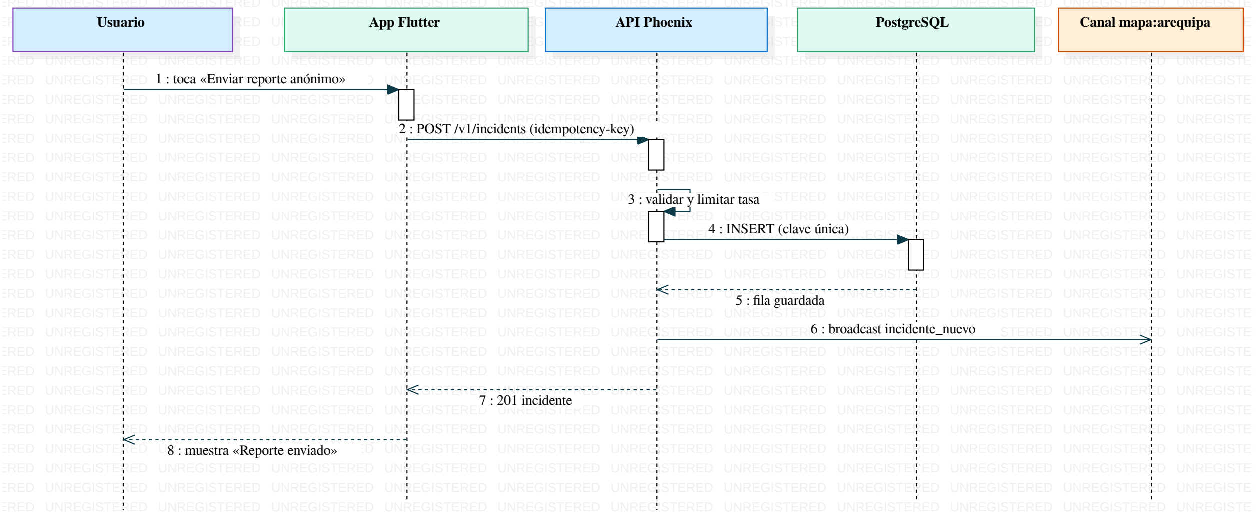

Usuario
App Flutter
API Phoenix
PostgreSQL
Canal mapa:arequipa
1 : toca «Enviar reporte anónimo»
2 : POST /v1/incidents (idempotency-key)
3 : validar y limitar tasa
4 : INSERT (clave única)
5 : fila guardada
6 : broadcast incidente_nuevo
7 : 201 incidente
8 : muestra «Reporte enviado»

**Figura 10:** Secuencia: enviar un reporte anónimo. Si el servidor responde 200 (repetición con la misma clave) no se emite ningún evento.

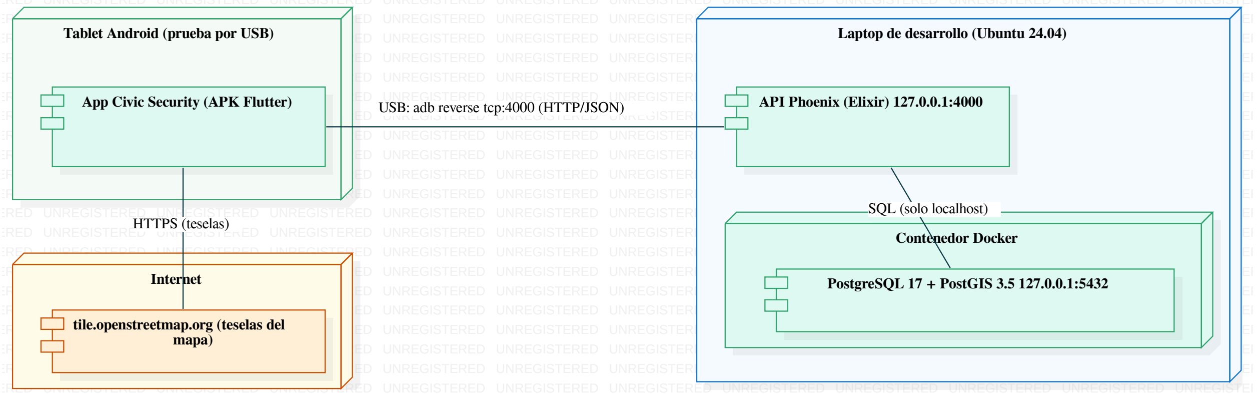

Tablet Android (prueba por USB)
App Civic Security (APK Flutter)
USB: adb reverse tcp:4000 (HTTP/JSON)
Laptop de desarrollo (Ubuntu 24.04)
API Phoenix (Elixir) 127.0.0.1:4000
HTTPS (teselas)
SQL (solo localhost)
Contenedor Docker
Internet
tile.openstreetmap.org (teselas del mapa)
PostgreSQL 17 + PostGIS 3.5 127.0.0.1:5432

**Figura 11:** Despliegue del entorno de pruebas: la tablet se conecta por USB y `adb reverse` lleva su puerto 4000 al servidor de la laptop, que sigue cerrado a la red.

con el backend local; si la medición lo justifica, se limitará y agrupará los pines dibujados y se simplificarán sombras, para que la app funcione con fluidez en dispositivos de distintas capacidades.

2. **Distancia (“a 660 m de ti”) que parecía corta.** La fórmula de la app coincide con el cálculo exacto de PostGIS (`app/test/core/distance_test.dart`: 3 208 m Plaza de Armas–Alto Selva Alegre, 431 m a la UCSP). Es distancia en **línea recta**, no por calles, y puede diferir de lo que se aprecia si la **ubicación del dispositivo** es imprecisa (una tablet sin GPS ubica por Wi-Fi, con errores de cientos de metros). *Acción:* mostrar en el mapa un círculo con la precisión de la ubicación.

## 9. Plan para la próxima semana

| Prio. | Actividad | Resultado esperado |
|---|---|---|
| 1 | **Integración continua** (GitHub Actions): pruebas, análisis estático, auditoría de dependencias y búsqueda de secretos en cada cambio | Cada cambio queda con marca verde o roja |
| 1 | **Tiempo real en la app**: escuchar `incidente_nuevo` y reconectar pidiendo el snapshot | Un reporte aparece en otro teléfono sin recargar |
| 1 | **Backend de cuentas**: sesión de invitado, registro con alias y contraseña (Argon2id), verificación por DNI con HMAC, límites por cuenta | Reportar y votar con cuenta |
| 2 | **Pantallas de Figma** 01–04 (bienvenida, login, registro, recuperación) conectadas a las cuentas | Flujo de registro funcional |
| 2 | **Rendimiento:** medir en modo *profile* en la tablet y optimizar; círculo de precisión de la ubicación | Mapa fluido en distintos dispositivos |
| 3 | **Validación comunitaria**: votos “Lo confirmo” / “Es falso”, regla de 3 confirmaciones a menos de 1 km, transición de estados | Diferencial del producto funcionando |

## A. Código representativo

**Listing 2:** Migración de la tabla `incidentes` (`backend/priv/repo/migrations/20261027172454_create_incidentes.exs`).

```
create table(:incidentes, primary_key: false) do
  add :id, :binary_id, primary_key: true
  add :tipo, :string, null: false
  add :descripcion, :string, size: 500
  add :ubicacion, :"geography(Point,4326)", null: false
  add :estado, :string, null: false, default: "sin_verificar"
  add :idempotency_key, :binary_id, null: false
  timestamps(type: :utc_datetime_usec)
end
create unique_index(:incidentes, [:idempotency_key])
create index(:incidentes, [:ubicacion], using: :gist)
create index(:incidentes, [:inserted_at])
```

**Listing 3:** Consulta del mapa (`backend/lib/civic/incidents.ex`).

```
  def list_recent({min_lng, min_lat, max_lng, max_lat}, %DateTime{} = since) do
    Incident
    |> where([i], i.inserted_at > ^since)
    |> where(
      [i],
      fragment(
        "ST_Intersects(?, ST_MakeEnvelope(?, ?, ?, ?, 4326)::geography)",
        i.ubicacion,
        ^min_lng,
        ^min_lat,
        ^max_lng,
        ^max_lat
      )
    )
    |> order_by([i], desc: i.inserted_at)
    |> limit(^@max_resultados)
    |> Repo.all()
  end
```

**Listing 4:** Envío idempotente del reporte en la app (`app/lib/features/report/presentation/report_controller.dart`).

```
  Future<void> submit(NewIncident input) async {
    // Un doble toque, o enviar de nuevo algo ya enviado, no hace nada
    if (state.sending || state.created != null) return;

    if (input != _lastInput) {
      _key = _uuid.v4();
      _lastInput = input;
    }
    state = const ReportState(sending: true);

    try {
      final created = await ref
          .read(incidentRepositoryProvider)
          .create(input, idempotencyKey: _key);
      if (!ref.mounted) return;
      ref.read(mapIncidentsProvider.notifier).upsert(created);
      state = ReportState(created: created);
    } on ApiFailure catch (failure) {
      if (ref.mounted) {
        state = ReportState(failure: failure);
      }
    } on Object {
      if (ref.mounted) {
        state = const ReportState(failure: ApiFailure.unexpected());
      }
    }
  }
}
```